

Smart Inventory & Expiry Management System

Comprehensive Presentation Guide

Project Overview:

A professional-grade, desktop-based Inventory Management System built with Python (Tkinter) and MySQL. It features a complete transactional database (with ACID compliance), real-time Dashboard KPIs, automated FIFO (First-In-First-Out) sales processing, and a background daemon thread for proactive expiry monitoring.

Part 1: Architecture & Database Foundation (Member 1)

Role Focus: Explain "how the data is structured and protected."

Core Concepts to Explain:

- The Tech Stack:** Python (Backend + UI via Tkinter) and MySQL (Database engine).
- Database Schema (7 Tables):**
 - Dictionaries:** `categories`, `suppliers`, `products`.
 - Transactional:** `batches` (when stock arrives), `inventory` (current available stock per batch), `sales` (what left the system), `expiry_log` (audit trail for wasted stock).
- Data Integrity (The "Smart" part):**
 - We do not store "total stock" as a single number on a product.
 - We store *stock per batch*. This allows us to track exact manufacture/expiry dates and exact cost prices for different shipments of the exact same product.

Key Logic to Highlight:

- Triggers:** Mention `trg_after_batch_insert` (auto-creates an inventory row when a batch is added) and `trg_after_sale` (auto-deducts stock when a sale occurs).
 - Safety Guards:** Mention `trg_before_sale` which strictly blocks any attempt to sell an expired batch or sell more stock than is physically available.
-

Part 2: Backend Modules & FIFO Sales (Member 2)

Role Focus: Explain "how the system processes complex business logic."

Core Concepts to Explain:

- Python Modules Structure:**
 - Instead of writing all code in one giant file, the project is cleanly separated into modules (`stock_manager.py`, `sales_manager.py`, `reports.py`).
 - `db_config.py` acts as the single central hub for connecting to MySQL.
- The FIFO Algorithm (Crucial Feature!):**
 - FIFO means "First-In, First-Out".
 - When a sale of 100 units is made, the system doesn't just subtract 100 randomly.
 - It querying the database for the *oldest expiring batches* first.
 - If Batch A has 40 units and Batch B has 200 units, a sale of 100 units will completely drain Batch A (40), and then deduct the remaining 60 from Batch B automatically inside a safe SQL Stored Procedure (`sp_fifo_sale`).

Key Logic to Highlight:

- Stored Procedures:** Emphasise that complex logic (like FIFO) runs directly on the MySQL server using cursors. This ensures operations are atomic (they either fully succeed or fully roll back if an error occurs), preventing corrupted data.
-

Part 3: Expiry Management & Threading (Member 3)

Role Focus: Explain "what makes the inventory system proactive and smart."

Core Concepts to Explain:

- Automated Expiry Sweeping (`sp_check_expiry`):**
 - The system must prevent expired drugs/products from being sold.
 - A routine scans all inventory to find batches where `expiry_date < today`.
 - It moves these records into a permanent `expiry_log` (an audit trail representing "Waste") and zeroes out the inventory so they can no longer be sold.
- The Background Daemon Thread (`scheduler.py`):**
 - To make this fully automatic, the application spawns a lightweight background "Daemon Thread" when it launches.
 - This thread uses the `schedule` library to quietly check for expiries at midnight every day without freezing the main user interface.
 - If it finds expired stock, it pushes an alert back to the main UI thread.

Key Logic to Highlight:

- Reporting:** Explain how `reports.py` generates the "Expiring Soon (within 7 days)" list. This allows staff to discount or prioritize selling goods before they expire, minimizing waste.
-

Part 4: The Graphical User Interface (GUI) (Member 4)

Role Focus: Explain "how the user interacts with the system and visually digests data."

Core Concepts to Explain:

1. Tkinter & Theming:

- Built using Python's built-in `tkinter` library, using `ttk` themed widgets.
- We implemented a custom dark mode color palette (backgrounds, interactive accent tracking) to give it a modern, professional Software-as-a-Service (SaaS) feel.

2. Application Flow (`main.py` -> `app.py`):

- `main.py` performs a mandatory Database Connection Check before launching the UI. If the database is offline, it fails gracefully with a helpful popup.
- `app.py` manages a `Notebook` (tabbed view) containing 5 distinct screens.

3. The 5 Tabs & Auto-Sync:

- **Dashboard:** Real-time KPI cards (Total Stock, Low Stock count, Revenue) and an urgent Expiring Soon table.
 - **Inventory:** View physical stock, add new Products, Suppliers, and Batches.
 - **Sales:** Process FIFO sales and view revenue/profit margins.
 - **Expiry:** The permanent audit log of all waste.
 - **Reports:** Low stock alerts, Total Inventory Valuation, and summary analytics.
 - **Auto-Sync Mechanism:** Explain that switching between any of these tabs automatically triggers a `.refresh()` function, querying the DB so the UI is always showing 100% live data.
-

Presentation Execution Flow (How to Demo It Live):

1. **Member 1 (DB):** Open `schema.sql` briefly, show the ER Diagram, explain the separation of products vs batches.
2. **Member 4 (GUI):** Launch `py main.py`. Show the Dashboard. Explain the modern dark UI and tab structure.
3. **Member 2 (Backend/Sales):** Go to the Inventory tab. Click + `Add Batch` to add stock. Go to the Sales tab and process a sale. Show how it calculates profit dynamically in the table.
4. **Member 3 (Expiry/Thread):** Go back to the Dashboard. Point out the "Expiring in 7 Days" list. Provide a brief explanation of how the background thread sweeps for fully expired items automatically. Show the Expiry tab audit log.
5. **Team Conclusion:** Summarise how the combination of MySQL procedures, strict validations, threading, and a clean UI makes this a fully robust, business-ready application.